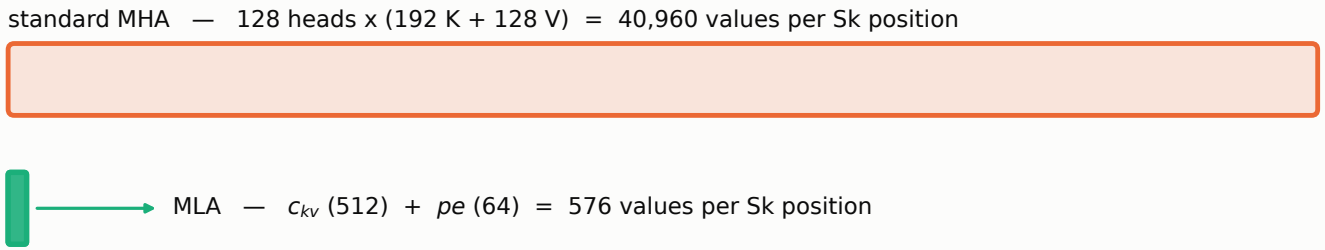
Why MLA: one shared latent instead of 128 K/V pairs

KV cache per token

Both bars count the values cached for ONE key position — a single index of Sk, shared by all 128 heads and by every one of the Sq queries.



The absorbed path: move the per-head matrix onto the query side

$q_{nope}[h]$: Sq x 128 $\quad$ c_{kv} : Sk x 512 $\quad$ $W_{uk}[h]$: 128 x 512 $\quad$ so $K_{nope}[h] = c_{kv} W_{uk}[h]^\top$: Sk x 128

$$q_{nope}[h] \cdot K_{nope}[h] \;=\; q_{nope}[h] \cdot \left(c_{kv} W_{uk}[h]^\top\right) \;=\; \left(q_{nope}[h]\, W_{uk}[h]\right) \cdot c_{kv}$$

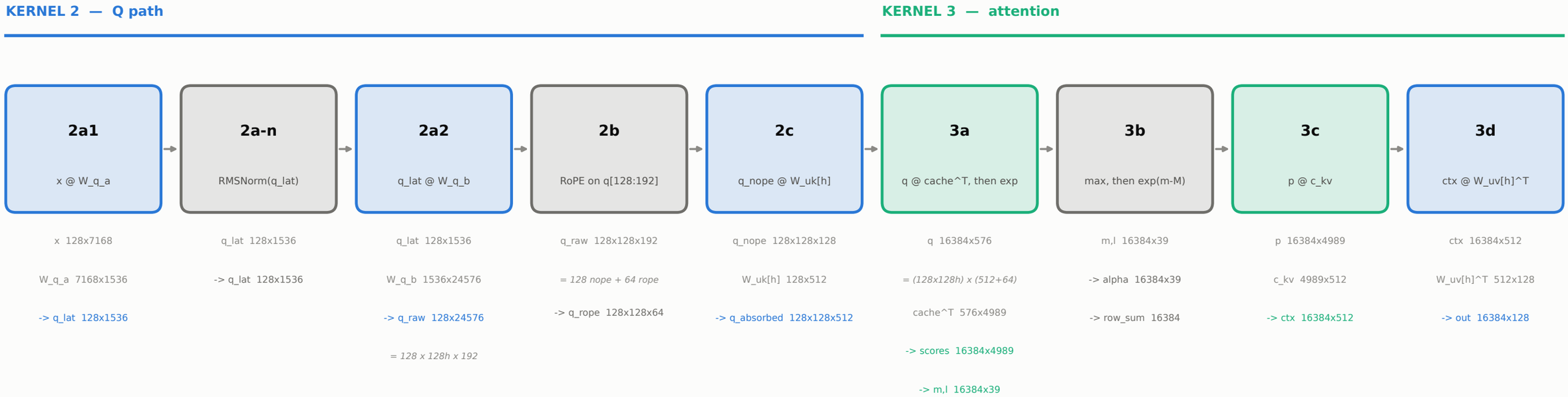
reconstruct K for all 128 heads
Sk x 128 x 128 values
grows with the cache

absorb into the query, ONCE
Sq x 128 x 512 values
Sq = 1 in decode

c_kv and pe_cache carry no head index: all 128 heads of a query read the SAME cache rows, so kernel 3a gives one block all 128 heads at once.

The 9 steps

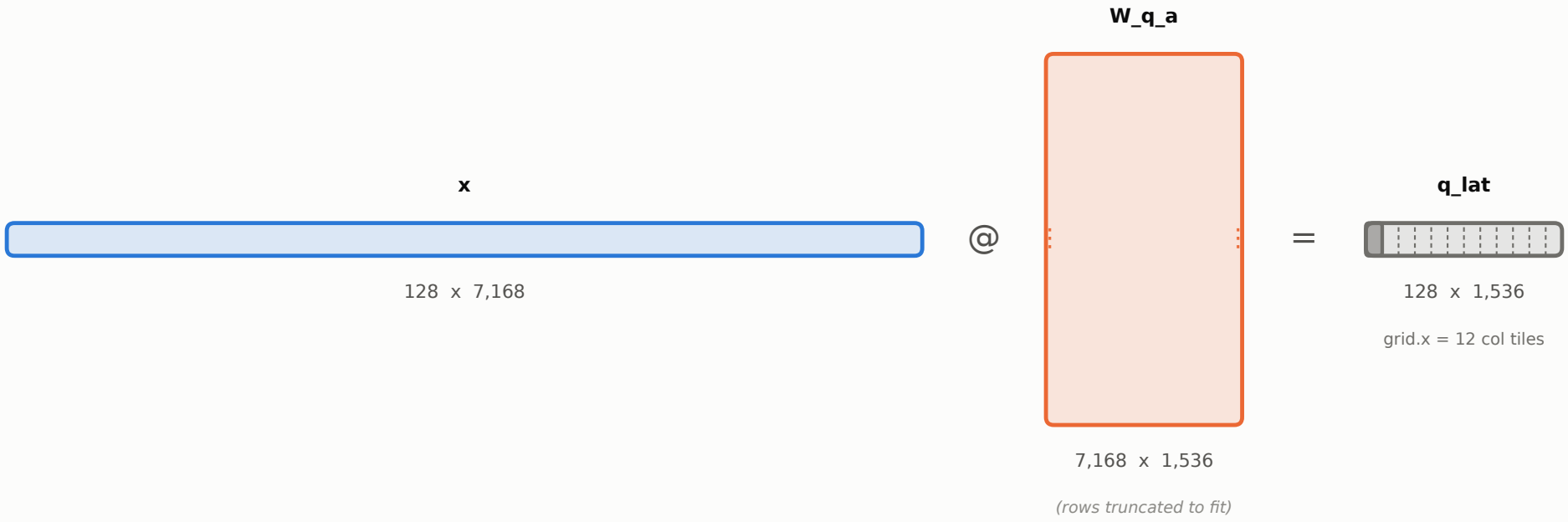
Kernel 2 builds the 576-wide query; kernel 3 does the attention. Shapes for batch-128 decode: bq = B.Sq = 128 rows, flat = B.Sq.H = 16,384 rows, Sk = 4,989. Every operand is listed by name; outputs are marked ->.



Step 2a1 — project the hidden state into the q_lora latent

q_lat = x @ W_q_a

first half of the factorised Q projection. 7,168 = hidden. 1,536 = q_lora_rank.



grid.x -> 12 output-column tiles of 128 | grid.y -> 1 row tile (bq = 128) | grid.z -> split-K: the 7,168 reduction is cut into slices
Only 12 column tiles means 12 blocks on an 84-SM GPU — split-K is what makes this step use the machine at all.

Step 2a-norm — RMSNorm on the latent

q_lat := q_lat * rsqrt(mean(q_lat^2) + eps) * gain

row-wise; keeps the two matmuls from collapsing into one rank-1536 matrix

q_lat (in and out)

128 x 1,536

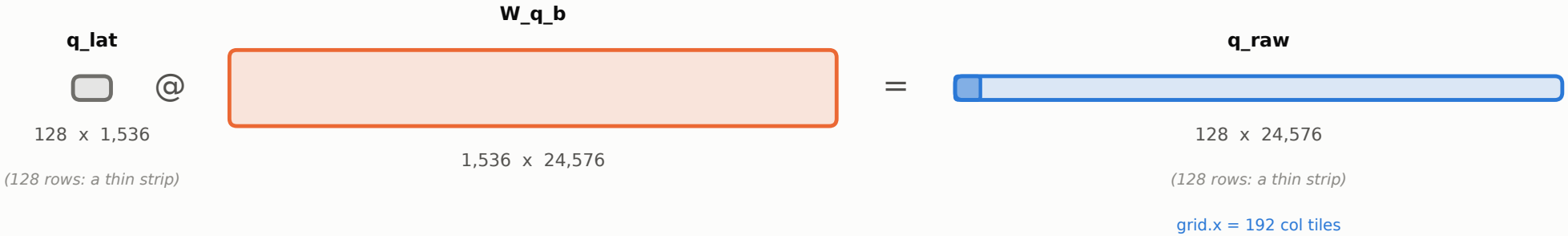
one CUDA block per row -> 128 blocks
256 threads cover 1,536 elements (6 each)
block-wide sum by warp shuffles, then one pass over the per-warp partials

Reads 2a1's wide fp32 accumulator and writes the narrow input dtype, which is what lets step 2a2 reuse the same GEMM kernel unchanged.

Step 2a2 — expand the latent into the per-head query

q_raw = q_lat @ W_q_b (24,576 = 128 heads x 192)

the second half of the factorised Q projection — the largest weight in the model at 151 MB

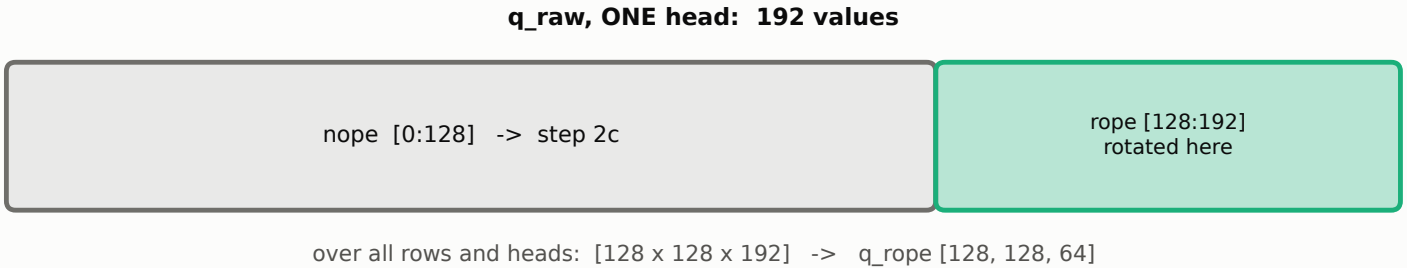


grid.x -> 192 output-column tiles of 128 | grid.y -> 1 row tile | grid.z -> split-K over the 1,536 reduction
192 blocks against 84 SMs x 2 resident = 168 slots is 1.14 waves: one full wave, then a tail wave leaving 144 slots idle.

Step 2b — RoPE on the rope half of each head

q_rope[b,s,h,:] = rotate(q_raw[b,s,h, 128:192], position[s])

elementwise; rotates adjacent pairs by an angle set by the query's ABSOLUTE position

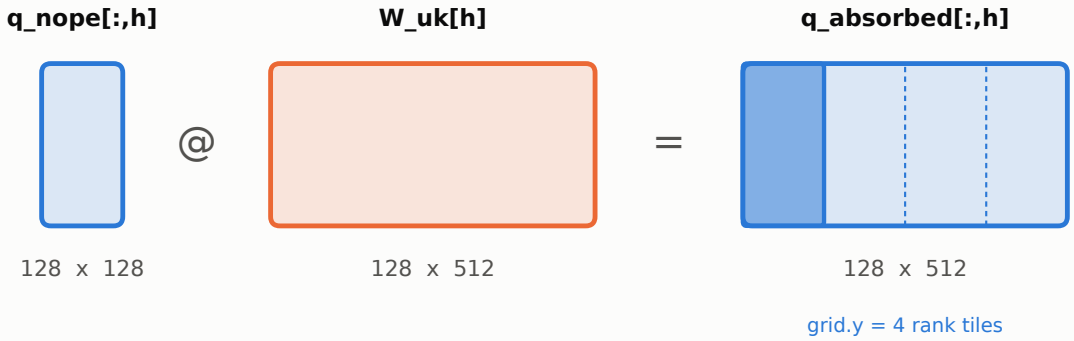


One thread per (row, head, adjacent pair), grid-stride capped at 32 x SM so the launch never scales with the cache depth. 0.1% of decode.

Step 2c — absorb W_uk into the query (once per head)

q_absorbed[:,h] = q_raw[:,h, 0:128] @ W_uk[h] repeated for h = 0 .. 127

this is the absorption: it widens the query 128 -> 512 so the cache can be read in its stored latent form

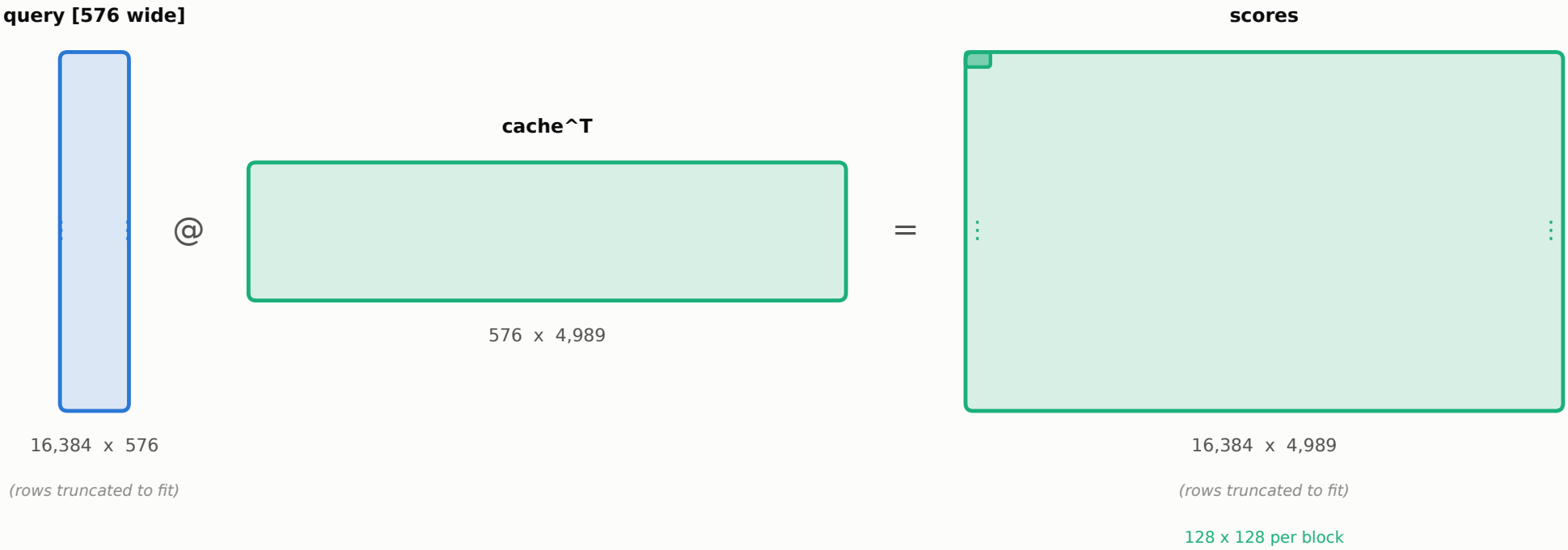


grid.x -> bq row tiles | grid.y -> 4 tiles of 128 over kv_rank=512 | grid.z -> the head index, 0..127
One head per block means W_uk[h] (262 KB) is staged into shared memory once and reused by every row in the tile.

Step 3a — scores, then the local softmax numerator

scores = (q_absorbed . c_kv + q_rope . pe_cache) / sqrt(192) then store exp(s - m_j)

the two dot products (512 and 64 deep) are run as ONE 576-deep reduction; shown at Sk = 4,989

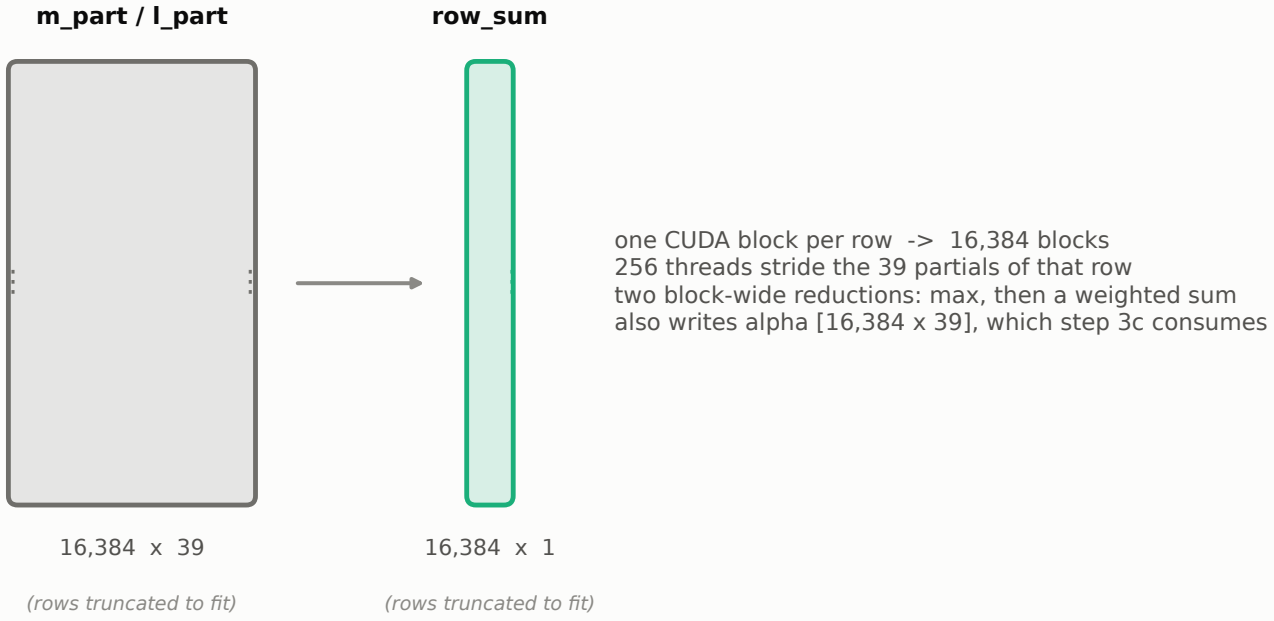


grid.x -> flat = B.Sq.H / 128 = 128 tiles | grid.y -> Sk / 128 = 39 tiles | no grid.z (scores has a real Sk axis)
A block's 128 rows are the 128 heads of ONE query, so the cache tile it stages is reused by all 128 of them.

Step 3b — reconcile the per-block softmax maxima

M = max_j m_j alpha_j = exp(m_j - M) row_sum = sum_j l_j * alpha_j

3a's grid.y split each row across 39 blocks, none of which could see the global maximum



one CUDA block per row -> 16,384 blocks
256 threads stride the 39 partials of that row
two block-wide reductions: max, then a weighted sum
also writes alpha [16,384 x 39], which step 3c consumes

Exact because exp(s - m_j) * exp(m_j - M) = exp(s - M) termwise. Total storage 7.7 MB against a 327 MB scores tensor; costs 0.5% of decode.

Step 3c — context: the attention-weighted sum of the cache

ctx = (exp(s - m_j) * alpha_j) @ c_kv = exp(s - M) @ c_kv, unnormalised

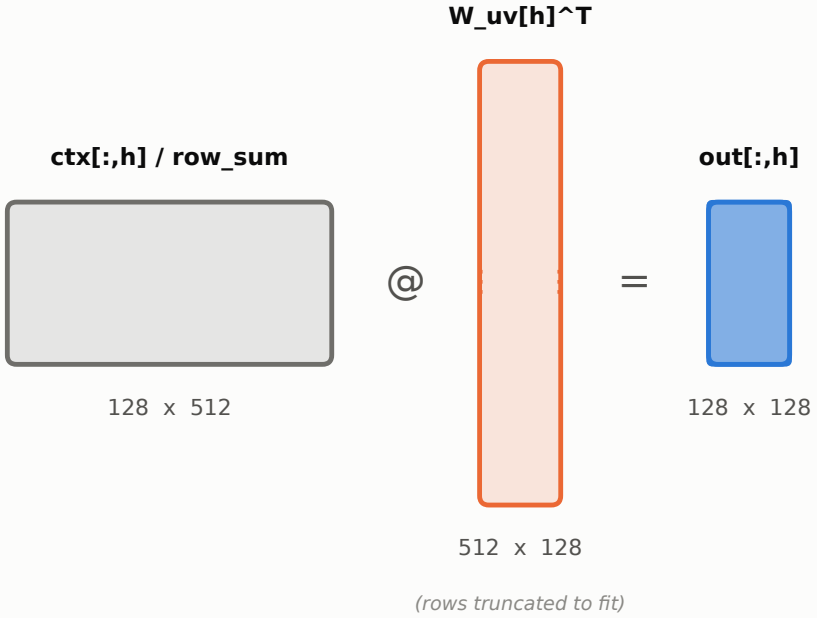
alpha_j is folded into the staging load, so what lands in shared memory is already the softmax numerator

grid.x -> flat / 128 = 128 tiles | grid.y -> 4 tiles over kv_rank = 512 | grid.z -> split-K over Sk
ctx has no Sk axis, so at B=Sq=1 the output alone yields 4 blocks and ~95% of the SMs would idle — hence split-K here.

Step 3d — reconstruct V and project out (once per head)

out[:,h] = (ctx[:,h] / row_sum) @ W_uv[h]^T repeated for h = 0 .. 127

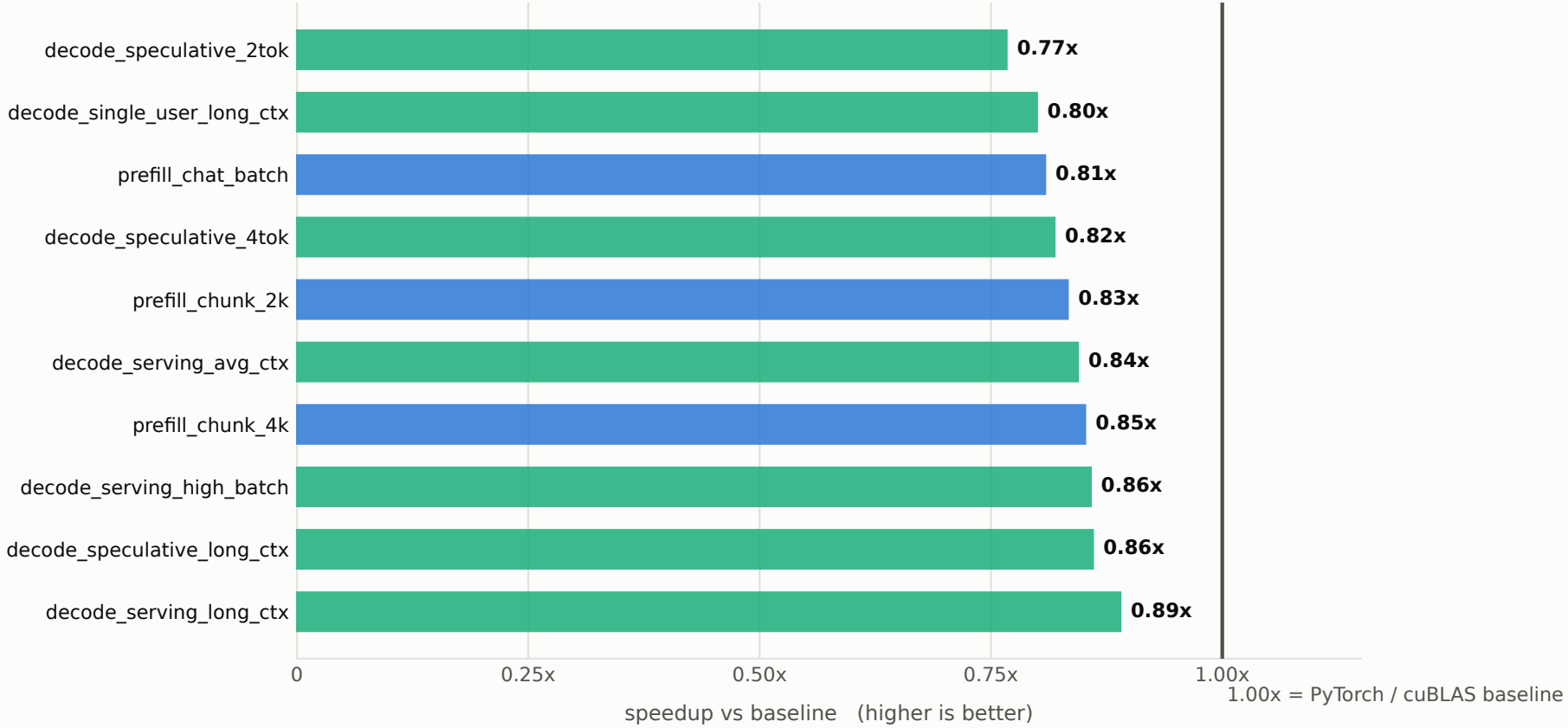
the deferred softmax divide rides along on a staging load this kernel was already doing



grid.x -> bq row tiles | grid.y -> 1 tile over v_dim = 128 | grid.z -> the head index, 0..127
Narrows 512 -> 128, undoing the widening that step 2c did. One head per block, same reuse argument as 2c.

End-to-end result: 0.83x geometric mean

FP32, RTX A6000, L2 flushed between iterations, TF32 disabled on both sides. All 10 scenarios pass correctness.



Decode (aqua) and prefill (blue) land in the same narrow band: the tiling generalises, but cuBLAS keeps a ~17% edge.